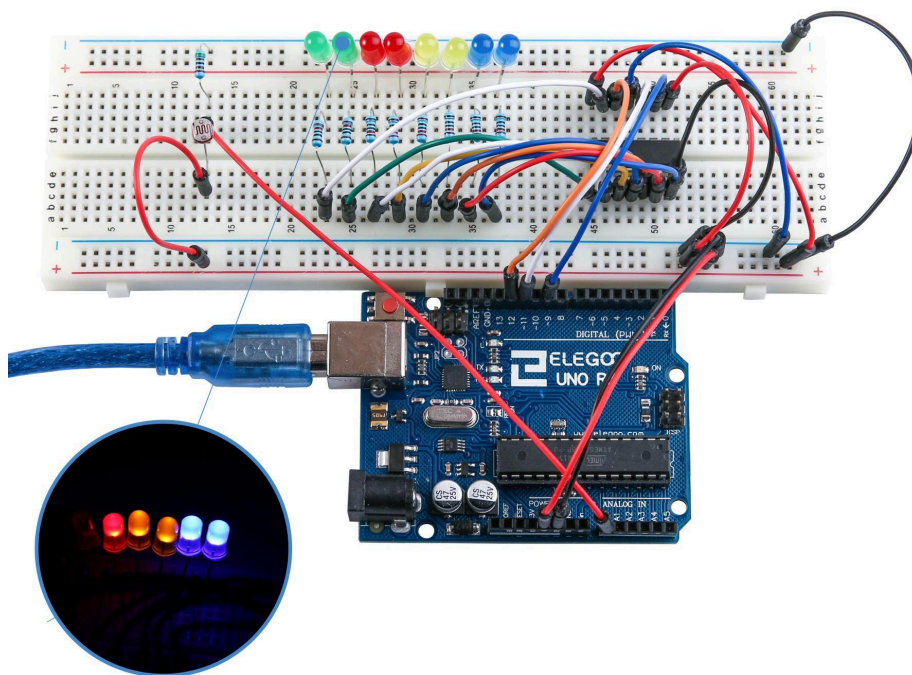


MODUL 5

Integration · Hackathon

Alla fyra modulerna, en krets, ett fungerande tjuvlarm.



Den typ av sammansatt krets ni byggde på hackathonen — RGB-LED:er, fotocell, buzzer, sammanvävda på en enda breadboard. Här en variant där fotocellen styr LED:erna direkt.

Vad du gjorde i dag

Sista träffen är en sammansmältning. Ingen ny teori, ingen ny syntax — allt ni behövde kunde ni redan. I stället satte ni ihop bitarna ni byggt separat och gjorde ett fungerande system av dem.

Systemet ni byggde:

- **En knapp** (Modul 3) som togglar ett ”larm-läge”.
- **En tilt-sensor** (Modul 4) som upptäcker om larmet rörs.
- **En fotocell** (Modul 4) som mäter omgivningsljus.
- **En RGB-LED** (Modul 2) som lyser stämningsljus i mörker.
- **En buzzer** (Modul 3) som tjuuter vid tilt-event.
- **Ett program** som binder ihop alltihop.

Reglerna var enkla, men programmet som realiserade dem krävde att varje bit från de föregående fyra modulerna sammanvävdes till en enda loop:

1. Knappen togglar `larmPaslaget` med **edge-detection**.
2. Om `larmPaslaget && tilt-sensorn rörs` → **tjut + rött blink**.
3. Om `!larmPaslaget && fotocellen är mörk` → **stämningsljus**.
4. Annars → **tyst och mörkt**.

Det är i princip ett minimalt embedded-system. Ni har byggt ett sådant. Grattis — ni kan nu beskriva grunderna för allt från disktermometrar till enkla RC-bilar i samma ordlista.

Systemets arkitektur

PIN-TILLDELNING

Samma pinnar som i varje lektion — återanvänds direkt:

<code>knappPin</code>	D9 — från Modul 3
<code>tiltPin</code>	D2 — från Modul 4
<code>ldrPin</code>	A0 — från Modul 4
<code>buzzerPin</code>	D12 — från Modul 3
<code>ledR</code>	D6 — från Modul 2
<code>ledG</code>	D5 — från Modul 2
<code>ledB</code>	D3 — från Modul 2

Alla dessa pinnar är PWM (~ -markerade) för R/G/B-kanalerna så att ni kan variera ljusstyrkan. Digital2 och Digital12 är vanliga digital-pinnar — inga krav på PWM där.

SENSE-ACT-LOOPEN

Strukturen för hela programmet är densamma som alla embedded-system någonsin skrivna:

1. **Läs av omvärlden** — fotocell, tilt, knapp.
2. **Uppdatera internt tillstånd** — hantera edge-detection för knappen.
3. **Bestäm vad som ska hända** — if/else baserat på `larmPaslaget` och mätvärdena.
4. **Styr utvärlden** — skriv till RGB-LED och buzzer.
5. **Paus** — kort `delay` för att inte bränna CPU:n.
6. Börja om.

Skelettet finns i slidesen; den kompletta lösningen finns i Bilaga D.

Tips för hackathon-formatet

SKRIV INTE ALLT PÅ EN GÅNG

Den vanligaste nybörjarfällan är att försöka skriva hela programmet direkt. Bygg **inkrementellt**:

1. Börja med att bara läsa knappen och printa till Serial Monitor varje gång den trycks. Bekräfta att edge-detection fungerar.
2. Lägg till `larmPaslaget`-togglingen. Printa `larmPaslaget` varje gång det ändras. Bekräfta att den flippar.
3. Lägg till tilt-sensorn. Printa ett meddelande när den lutar. Ignorera ännu allt med larm-läget.
4. Lägg till villkoret "om larm på + tilt lutad → `digitalWrite(buzzerPin, HIGH)`". Testa.
5. Lägg till fotocellen. Printa värdet. Hitta din tröskel.
6. Lägg till stämningsljuset.
7. Finslipa.

Mellan varje steg: **ladda upp, testa, bekräfta att det nya steget fungerar innan du går vidare**. När något går sönder vet du exakt vad det är — det är det du nyligen la till.

SERIAL MONITOR ÄR INTE VALFRI

Varje gång du lägger till ett nytt villkor eller en ny variabel, printa den. Printa `larmPaslaget`. Printa `ljus`. Printa `state`. Printa varje gren av varje if/else. Om beteendet är mystiskt — läs vad Serial Monitor säger, inte vad du **tror** står i koden.

DEBUG-PRINT SNABBRECEPT

```
Serial.print("larm=");  
Serial.print(larmPaslaget);  
Serial.print(" ljus=");  
Serial.print(ljus);
```

```
Serial.print(" tilt=");  
Serial.println(tilt);
```

Kör det på slutet av varje loop (efter ett lämpligt `delay(200)`) så har du en kontinuerlig ström av sanning.

EDGE-DETECTION ÄR ICKE FÖRHANDLINGSBAR

För knappen: **alltid** edge-detection, aldrig direkt `if (digitalRead(knappPin) == LOW)` . Utan edge-detection hoppar `larmPaslaget` fram och tillbaka hundratals gånger per tryckning, och du har ingen aning om vilket värde den slutade på när du släppte.

För tilten: edge-detection är mer valfritt. Ni kan agera direkt på ”tilt är LOW nu”, och det fungerar oftast bra. Men samma problem med studs kan uppstå — lägg `delay(50)` efter läsningen.

ORDNA KODEN I BLOCK

Ett strukturerat sätt att skriva loopen:

```
const int morkTroskel = 300; // kalibrera själv  
  
void loop() {  
  // 1. LÄS INPUTS  
  int ljus = analogRead(ldrPin);  
  bool lutad = digitalRead(tiltPin) == LOW;  
  int state = digitalRead(knappPin);  
  
  // 2. UPPDATERA INTERNT TILLSTÅND (edge-detection)  
  if (state == LOW && lastState == HIGH) {  
    larmPaslaget = !larmPaslaget;  
  }  
  lastState = state;  
  
  // 3. BESTÄM UTFALL  
  if (larmPaslaget && lutad) {  
    // LARM – tjut + rött  
    digitalWrite(buzzerPin, HIGH);  
    analogWrite(ledR, 255);  
    analogWrite(ledG, 0);  
    analogWrite(ledB, 0);  
  } else if (!larmPaslaget && ljus < morkTroskel) {  
    // STÄMNINGSLJUS – mjukt varmvitt  
    digitalWrite(buzzerPin, LOW);  
    analogWrite(ledR, 120);  
    analogWrite(ledG, 60);  
    analogWrite(ledB, 20);  
  } else {  
    // TYST OCH MÖRKT  
    digitalWrite(buzzerPin, LOW);  
  }  
}
```

```

    analogWrite(ledR, 0);
    analogWrite(ledG, 0);
    analogWrite(ledB, 0);
  }

  // 4. KORT PAUS
  delay(10);
}

```

Tröskeln 300 i exemplet är **bara en gissning**. Kalibrera själv med Serial Monitor innan hackathonen börjar — värdet hör hemma som en `const int` i toppen så du enkelt kan ändra det.

Vanliga problem och snabblösningar

Larmet flimrar när knappen hålls	Du saknar edge-detection. Återvänd till "Reagera på flanken" i Modul 3.
Tilten triggar slumpmässigt	Studs. Lägg <code>delay(50)</code> efter <code>digitalRead(tiltPin)</code> .
RGB-LED blir aldrig riktigt mörk	Glömt att sätta alla tre kanalerna till 0 i "tyst och mörkt"-grenen. Alla tre kanaler ska vara <code>analogWrite(pin, 0)</code> .
Stämningsljuset lyser alltid	Din tröskel är för hög — <code>ljus < 300</code> matchar även rumsljus. Sänk till 150 och testa.
Buzzern vägrar bli tyst	En av grenarna sätter <code>HIGH</code> men ingen annan återställer till <code>LOW</code> . Säkerställ att varje gren av <code>if/else</code> skriver både buzzer och LED.
Kompilerar inte — "variable not in scope"	Variabler deklarerade inne i <code>setup()</code> är bara synliga där. Flytta <code>lastState</code> och <code>larmPaslaget</code> upp till global nivå (utanför <code>setup</code> och <code>loop</code>).

Bygg vidare

Hackathonen ger en fungerande grund. Om ni vill vidare — allt ni behöver är redan i kittet:

- **Dubbelt pip för larm-på/larm-av**. Ett kort pip när larmet aktiveras, två snabba när det stängs av. Förtydligande till användaren vad som hände.
- **Dimning i stället för hopp**. När stämningsljuset tänds — tona upp det över en sekund i stället för att bara slå på. `for`-loop med stegvis ökande `analogWrite`.
- **Hysteres**. Dela tröskeln i två: en "tänd vid 300, släck vid 400". Det hindrar LED:en från att blinka av och på när ljuset ligger precis på tröskeln.

- **Förvarning.** När larmet varit på i 30 sekunder, blinka buzzerN i ett mönster som varning om att du glömde stänga av den.
- **Eget morse-meddelande** från pin 13 varje gång tilten triggar.

Vilken som helst av dessa är 15–30 rader extra kod. Prova.

Snabbreferens

Pin-karta	Knapp D9, Tilt D2, LDR A0, Buzzer D12, R/G/B D6/D5/D3.
Kod-skelett	setup: <code>Serial.begin</code> + <code>pinMode</code> för alla. loop: läs, edge-detect, if/else, styr.
Grundläggande debug	<code>Serial.print(variabel)</code> på allt som är mystiskt.
<code>!larmPaslaget</code>	Logisk inversion — togglar <code>true</code> / <code>false</code> .
Full sketch	Se Bilaga D.

Efter kursen

Fem veckor sedan visste ni inte vad en GND-pinne var. I dag har ni byggt ett system som läser av omvärlden och reagerar på den. **Det är i princip vad embedded-ingenjörer gör på jobbet** — bara i mindre skala och med billigare komponenter.

Nästa steg — om ni vill fortsätta — är inte ett enda steg utan en skog av dem:

- **Arduino-projekt online.** Sidor som Hackster, Instructables, Make: Magazine är fulla av recept för allt från väderstationer till MIDI-instrument. Börja enkelt. Kopiera något. Förstå vad du kopierade.
- **Lokala makerspaces.** Det finns ofta ett i närmsta större stad. Gå en kväll. Andras projekt är smittsamt.
- **Fråga er själva: vad i mitt eget hem skulle jag vilja automatisera?** Ett litet problem i verkligheten är mycket bättre motivation än en abstrakt tutorial. ”Lampan vid hallspegeln tänds när jag kommer hem i mörkret.” ”Brevlådan skickar en notis när posten kommit.” ”Kaffebyggaren pausar två minuter efter att jag stängt den av.” Bygg det lilla. Lär dig det stora på vägen.
- **Gå längre in i elektroniken.** De fyra operatorerna från Bilaga A räcker som programmerarpotential för nästan allt Arduino-relaterat. Begränsningen är snart inte koden — den är era idéer om vad som borde göras.

Om ni kommer tillbaka och visar upp något ni byggt: det är kursens högsta beröm. Detta kompendium är er utgångspunkt, inte er slutpunkt.